

CS Software Project

A dynamic referral search engine for Realtors and their clients

Jordan and Reece

2026-02-10

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Rodney Fontil
- Client title: Real Estate Agent
- Client email address:
- Client employer: Hearth Realty
- How you know the client: Cold outreach

2 Project Description

2.1 Overview

Real estate agents struggle to stay connected with clients after closing. Rodney reports that 80 percent clients don't return to their realtors when selling their next home or to refer friends. It matters to him because this is a missed opportunity to nurture that client relationship and for repeat business. The root cause is that agents fall out of clients' minds once the transaction ends.

To address this problem, Rodney is requesting a web platform that helps real estate agents maintain on-going engagement with past clients through automated, personalized touchpoints. The app will enable agents to stay top-of-mind by delivering timely, relevant content and communications that keep the relationship active long after the sale is complete.

2.2 Key Features

These are from most key to least important.

- User types: There will be 4 types of users, agents, clients, admins, guests.
- Clients, Agents and Organizations: Clients will be invited and associated with specific agents on the platform for more exclusive referrals, and even access to the agent's broader organization.
- Guests: Guests can view search engines agents and their organizations share to the public.
- Admins: Admins can manage referral databases for their branches. This is for the organizations who want all their agents to use the same database.
- Client interfaces: These interfaces are where users like clients can query their agents' referral engine, find public referral engines, and manage chats with their agent.

- Agent and Admin interfaces: These interfaces will house all the functionality for agents to invite clients, add and update their referral databases and chat with their clients.
- Natural language search: Clients can type questions to get matched with referrals ("good pediatrician near downtown?") rather than filtering through categories. Here we can use natural language processing and AI to make the user search experience more intuitive.
- Analytics dashboard: Shows agents what types of referrals clients request most often, helping them build stronger local networks
- Branch collaboration mode: Multiple agents in the same real estate office can pool their recommendations into one shared database, expanding the value for all their clients
- Lead tracking: When clients use the platform, agents see which services are being searched, helping them identify clients who might be ready to sell (e.g., someone searching for major contractors might be preparing their home for market)
- Agent / client chats: When questions can't be supported by a database query, clients can send their Agent a message on the platform.

2.3 Why this Project is Interesting

Because we get a chance to work on a real problem. We also get to tackle it using AI technology as it makes what would just be a simple search engine into a more dynamic intuitive experience for the end user — clients.

2.4 Areas of CS required

- Databases: Store agents' referral catalogs with business details, categories, and metadata. Support efficient queries across potentially thousands of entries as the platform scales
- Machine Learning: Power the natural language search using text embeddings that match client questions to relevant referrals, even when exact keywords don't match
- Object-Oriented Programming: Structure the codebase with classes for Users (Agents/Clients), Referrals, Searches, and Analytics to maintain clean, extensible code
- Software Architecture: Design the system to handle the branch mode feature where multiple agents share one database while maintaining separate analytics, requiring careful data separation
- User Experience (UX): Create an interface simple enough that clients will use it spontaneously when they need a referral, not just when the agent asks them to
- Cybersecurity: Protect sensitive search data (e.g., if a client searches for divorce lawyers or therapists, that information must remain private)
- Ethics: Give clients control over which searches are visible to their agent, balancing privacy with the agent's business need to understand client needs

2.5 Potential Concerns and Questions

Data privacy is the primary concern. Clients might search for sensitive services (therapists, family lawyers, medical specialists). The platform needs to let clients mark certain searches as private—hidden from their agent's analytics dashboard—while still returning helpful recommendations. This creates a technical challenge: how do we provide value to agents through search insights while protecting client privacy when they need it? We'll need to determine the right default setting: Are all searches visible to agents unless clients opt out? Or is everything private unless clients opt in? The answer affects both the platform's value proposition to agents and clients' trust in using it.

3 Requirements

3.1 Non-Functional Requirements

ID	NFR Title	Category	Description
02	private messages	Security	all messages clients, or quests share (even if saved) should be kept visible only to the sender since some questions could be personal. The only attribute that should be visible is the category of the query for analytical purposes.
03	RAG database management	Usability	Our system should be able to dynamically set up these rag system via a API. For this we can use AWS.
04	Branding customization	User-experience	Branding should be flexible. an agent may switch brokerages overtime, update their logo etc. Those small changes should be able to be done in the platform.
05	Search alternative	User-experience	A agent's data base may not have all the answers to a users question. In this event we should have a back up to pass the users question off to google API, or to send a inquiry to their Agent.
06	Authentication and permissions	Security	These will be used to ensure data privacy.

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

ID	Story Title	Points	Description
C1	Request Personalized Referral	5	As a Client, I want to message my agent when no referrals match my search so that I can request a personalized recommendation.
C2	Search Referrals	8	As a Client, I want to search for referrals using natural language questions (e.g., "Where can I get my roof fixed?") so that I can find relevant recommendations without needing to know exact categories or keywords.
C3	Basic referral search	2	As a Client, I want to search my agent's referral database for specific entries so that I can quickly find the recommendations I need.
C4	Save search history	5	As a Client, I want to save searches I have with my agents database so that I can quickly refer back to old questions I had.
A5	Invite clients	2	As an Agent, I want to send platform invites to my clients so that they can access my exclusive referral list and communicate with me directly.

A6	Client Analytics	5	As an Agent, I want to view and filter through analytics on my clients' searches so that I can understand their needs and identify gaps in my referral database.
A7	Referral Management	5	As an Agent, I want to create, edit, and delete referrals so that I can keep my recommendations current and relevant for my clients.
A8	Referral Edits	3	As an Agent, I want to edit referrals so that I can keep my recommendations current and relevant for my clients.
G9	Guest searches	3	As a Guest, I want to access public referral lists from agents and branches so that I can find local recommendations without creating an account.
A10	Exclusive Access to Referral Lists	2	As an Agent, I want to restrict my referral list to only my clients so that I can provide exclusive value to the people I serve.
U11	Log in and out	1	As a User, I want to log in and out of my account so that I can securely access my agent's personalized recommendations.
C12	Client account management	3	As a Client, I want to update my profile information and preferences so that my agent can provide more relevant recommendations.
AD13	Create a branch	2	I want to create a branch profile with our branding and office information so that clients can identify and connect with our team.
AAD14	Add new agents to a branch	2	As an Agent or Admin, I want to add agents to my branch so that they can manage their own client relationships and referral lists.
AD15	Branch database management	8	As an Admin, I want to create, update and delete a master referral database for my branch so that all agents have access to our office's recommended vendors and services.
A16	Google Search Relay	3	As an Agent, I want the system to provide Google search results when my database has no matches so that my clients always receive helpful information even for referrals I haven't added yet.
U17	Message inbox	5	As a User, I want to access an inbox so that I can receive and manage invites, messages, and notifications in one place. Emails should be sent to the recipient to ensure they know when they have new notifications
C18	Search filters	3	As a Client, I want to filter my searches by category, location, or service type so that I can find the most relevant referrals for my specific need.
AAD19	Custom Referral tags	5	As an Agent/Admin, I want to add, update, and delete tags on my referrals so that clients can filter results and find what they need more easily.
U20	Search suggestions	3	As a User, I want to select from suggested search questions using a algorithm so that I can find common referrals without typing.

Table 2: Functional requirements as User Stories.

4 System Design

4.1 Architecture

We are following the MVC (Model View Controller) architecture. Potential Main Modules:

- Client Management Module
- Referral Management Module
- Branch Management Module
- Analytics Module
- Admin Module
- Notification Module

4.2 Diagrams

Figure 1: Simplified class diagram for the referral page. More to follow over the course of the development.

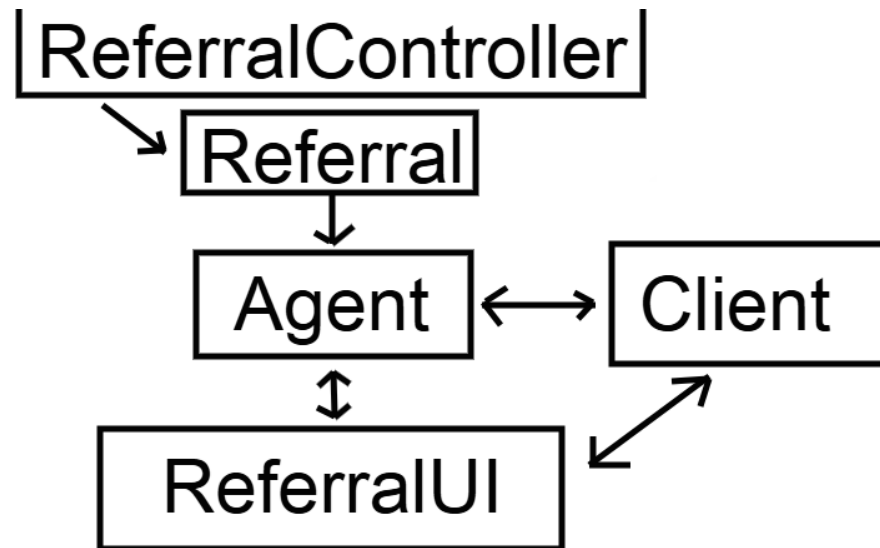


Figure 1: Referral Page Diagram

4.3 Technology

- Bubble.io — rapid front-end development and database functionality
- RAG databases (i.e. Chromadb)
- JavaScript / react and python for coding
- Docker to package our code for deployment
- Render or AWS for deployment

4.4 Coding Standards

4.4.1 Naming

- **Global Variables:** All caps and underscores for spaces. Examples: `VARIABLE`, `TEST`, `MIN_NUM`
- **Classes:** Capitalized and CamelCase. Examples: `Class`, `ClassGuide`, `ExTwo`
- **Functions:** Lowercase with underscores for spaces. Avoid using numbers. Examples: `function`, `function_two`, `print_all_one`
- **Variables:** Same as functions, but numbers do not require underscores. Examples: `n`, `x`, `x2`, `print_status`
- **Files:** File names should be descriptive and follow variable naming rules unless another convention is necessary. Examples: `test_file.txt`, `grid_layout.py`, `App.jsx`
- **Folders:** Same as files. Keep names general and simple. Examples: `folder`, `folder_two`, `src`

4.4.2 Code Formatting

- Include a blank line between each function.
- Use consistent and readable loop structures.
- Place comments above functions and significant code blocks.
- Inline comments should be aligned to the right of the code line they describe.
- Maintain consistency throughout the code base.

4.4.3 Commenting Rules

- Comments must be placed above each function explaining its purpose.
- Comments must be placed above each class.
- Comments should be detailed and easy to understand.
- Inline comments (for specific lines) must appear to the right of that line.
- Comments for code blocks should be placed above the block and explain its function.
- Create and follow standard templates for class and function comments.

4.4.4 Coding Principles

- Follow the **DRY (Don't Repeat Yourself)** principle when designing code.

4.4.5 Testing

- Our code will test with Jest and require above 80% coverage to be committed.

4.5 Data

The database structured around five main collections: Users, Conversations, Messages, Referrals, and Notifications.

The Users collection stores all account data for both agents and clients, including arrays referencing their conversations, clients, referrals, and notifications, along with profile information and preferences. These collections interact through ID references: users reference their conversations and notifications, conversations reference their messages and participants, and referrals connect multiple users together, creating a flexible relational structure that supports efficient querying while maintaining the scalability advantages of NoSQL.

(Diagram below)

4.6

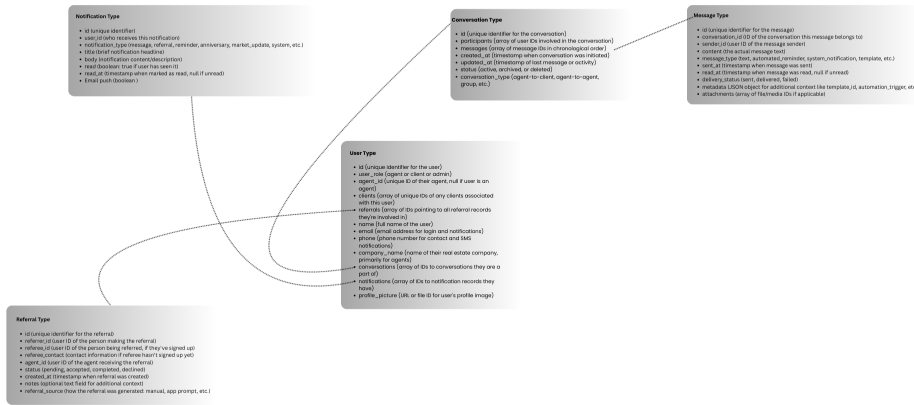


Figure 2: A diagram for our data types and how they connect

5 UI Mocks

5.1

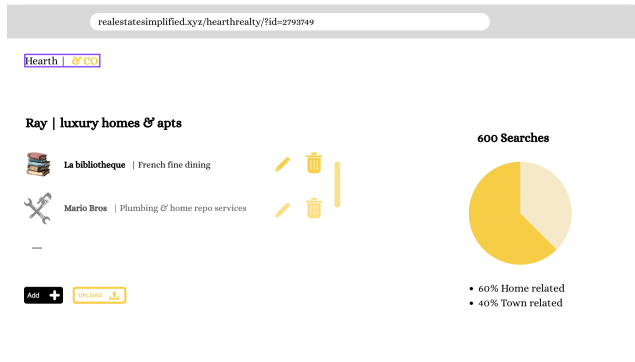


Figure 3: Mock up for the Agents view, all their referrals and a view to see analytics

5.2

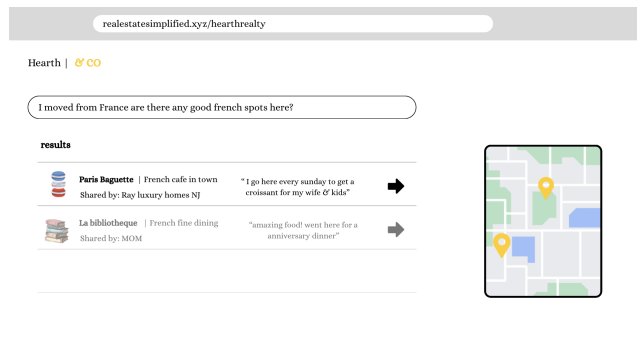


Figure 4: Mock up for how the search interface could look like

5.3

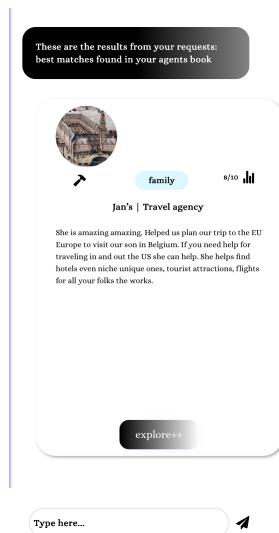


Figure 5: Another Mock up for how the search interface could look like, more like a AI chat user interface

6 Iterations

6.1 Iteration Planning

6.2 Iteration Planning

6.3 Iteration/Sprint 1

6.3.1 Planning

The primary goal for this iteration was the initial software environment setup (Render, Bubble, GitHub, etc.) followed by implementing core user logic. A key focus was ensuring seamless connectivity between Bubble and our server to lay the groundwork for future iterations. We aimed to make all endpoints accessible to facilitate the transfer of user data, referrals, and other database objects from Bubble to our server for processing.

During this sprint, we targeted the following features:

Iteration	Dates	Stories
1	02/10/2026	A5 1 C12 gin/L
2	02/24/2026	C3 B agem Acces
3	03/17/2026	C2 S A6 C
4	03/31/2026	C1 R ate a AD15
5	A16 Google Search Relay, C18 Search Filters, AAD19 Custom Referral Tags, U20 Search Suggestions	
total 70		14

Table 3: Iteration Planning for Incremental Deliveries

Authentication: User Login and Logout logic.

Account Management: Basic profile settings and invite functionality.

Communication: A functional message inbox.

UI/UX: Developing the necessary front-end components to support these features.

6.3.2 Work Done

We developed four core pages within the Bubble UI:

Index/Landing Page: The initial user entry point.

Navigation Hub: A post-login redirect page to route users to either the Agent or Client interfaces.

Agent UI: A dedicated dashboard for agent-specific tasks.

Client UI: A dedicated dashboard for client-specific tasks.

On the backend, we successfully deployed our server on Render, implementing:

User Authentication logic (Sign-up/Log-in).

Bubble API connector functions for data synchronization.

CRUD operations for referrals (initial implementation).

A technical spike exploring natural language search for a home page prototype.

6.3.3 Testing Coverage

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	87.8	63.63	100	87.8	
API_bubble	100	100	100	100	
api_connect.js	100	100	100	100	
user_logic	86.84	63.63	100	86.84	
users.js	86.84	63.63	100	86.84	79,141-149

Figure 6: Testing Coverage for Iteration 1

6.3.4 Retroespective & Reflection

The initial phase was marked by several technical "hiccups," particularly regarding software constraints. We encountered unexpected paywalls affecting collaboration on Bubble and initial configuration friction on Render. Furthermore, we identified a need for more frequent communication to align our workflows.

Despite these delays, I am confident in our tech stack. The integration between Bubble.io and Render creates a highly dynamic development environment; our CI/CD pipeline ensures that as soon as we commit to GitHub, Render deploys the server, making custom code immediately accessible to Bubble via API.

Bubble has significantly accelerated our front-end progress. In previous projects, the lack of immediate visual feedback when coding UIs in an IDE slowed development. Bubble's component library and native database integration remove that friction. But there were collaboration limitations — our current plan to work around this is adopting a pair-programming for our in person meetings. We also discovered that using the "test-version" allows both of us to view front-end changes in real-time, ensuring visibility for both team members.

In the future more "homework" on platform paywalls and collaboration tiers is essential. This iteration highlighted the importance of full-stack visibility; for a project to move efficiently, every developer must understand how to access the database, trigger deployments, and navigate the integrated environment.

6.4 Iteration/Sprint 2

6.4.1 Planning

6.4.2 Work Done

6.4.3 Testing Coverage

6.4.4 Retroerspective & Reflection

6.5 Iteration/Sprint 3

6.5.1 Planning

6.5.2 Work Done

6.5.3 Testing Coverage

6.5.4 Retroerspective & Reflection

6.6 Iteration/Sprint 4

6.6.1 Planning

6.6.2 Work Done

6.6.3 Testing Coverage

6.6.4 Retroerspective & Reflection

6.7 Iteration/Sprint 5

6.7.1 Planning

6.7.2 Work Done

6.7.3 Testing Coverage

6.7.4 Retroerspective & Reflection

7 Final Remarks

7.1 Overall Progress

7.2 Project Reflection

Appendix